Testování softwaru

Přednáška 2:
Techniky pro návrh testovacích scénářů:
kombinace vstupů

Miroslav Bureš 2016

Základní pojmy

Smoke test

Sada základních testů prověřujících nejdůležitější funkcionalitu systému. Cílem je ověřit, že nejpodstatnější části aplikace fungují.

Synonyma: Sanity test, Confidence test

Test condition

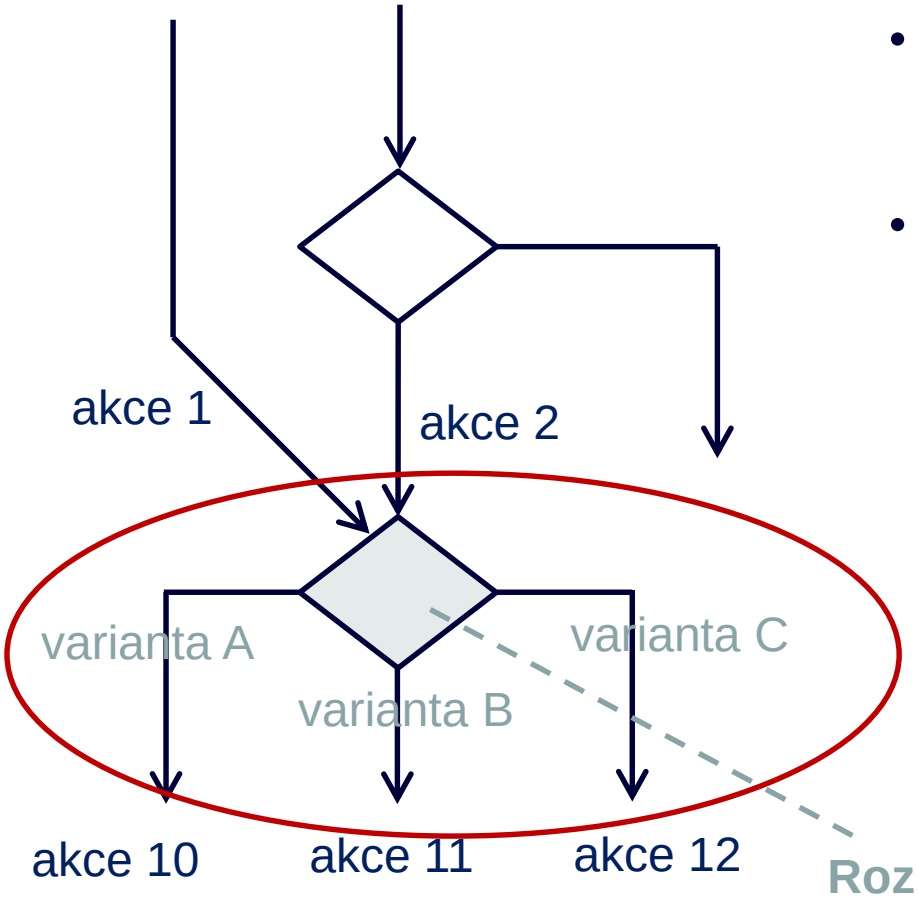
(Obecný) element nebo událost v systému, kterou je možné prověřit jedním nebo více testovacími případy. Například: funkce, transakce, strukturální element

Synonymum: **Test situation**

+ testovací pokrytí z první přednášky

Úvod

- **Představte si možné akce a přechody v testované aplikaci jako UML activity diagram, pro zjednodušení pak jako orientovaný graf**



- Základ pro testovací scénář: Vyvolej funkci aplikace s nějakou kombinací vstupních dat
- Kombinace vstupních dat v rozhodovacím bodě určují, jaká další akce se v aplikaci vykoná

Jak vhodně nakombinovat hodnoty (data) ve vstupech pro rozhodovací bod?

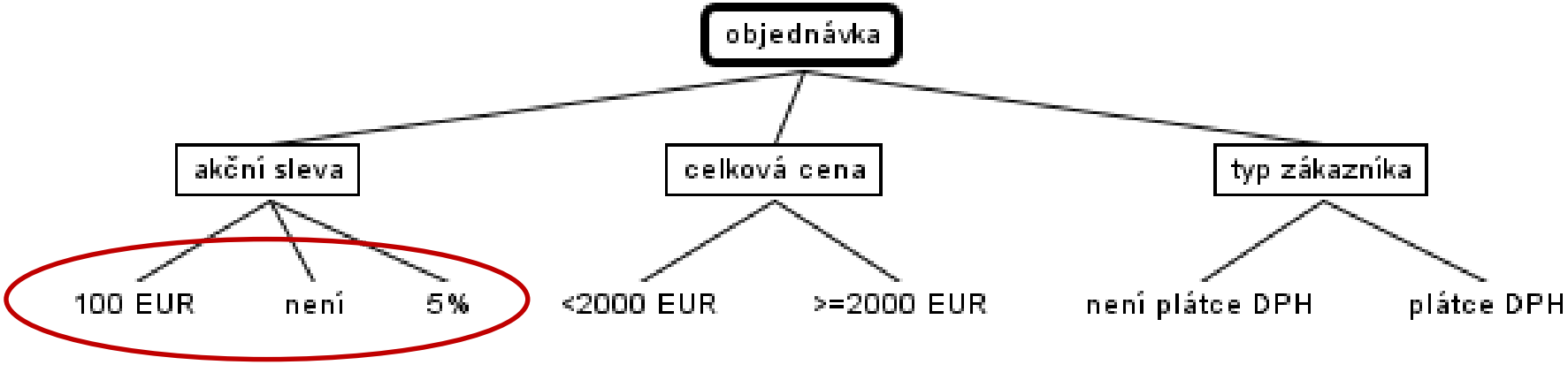
(nejdřív se budeme věnovat tomuto, pak budeme kombinovat sekvence akcí za sebou)

Rozhodovací bod
Test condition

Klasifikační strom

- Můžeme vytvořit pro každý rozhodovací bod v aplikaci
- Dá se aplikovat na různých úrovních: vstup dat přes webový formulář, interní výpočet a rozhodování v aplikaci, test integračního rozhraní, …
- **Uzly = classification ~ jednotlivé vstupy pro rozhodovací bod** (příklad: políčka z webového formuláře, část podmínky v procesu rozhodování, …)
- **Listy = třídy ekvivalence** pro danou classification (viz dále)

- Příklad:



Jak určíme, jaké hodnoty (data) zadávat do konkrétního vstupu?

Třída ekvivalence

„Do vstupu můžu zadat velké množství hodnot. Jak vybrat vhodnou množinu pro test?“

- Equivalence class nebo Equivalence partition, dále zkratka EC
- Konkrétní množinu hodnot, které jsou zadat do vstupu, rozdělíme na podmnožiny - třídy ekvivalence:

- Systém se má podle specifikace (na základě které testujeme) pro všechny hodnoty z konkrétní třídy ekvivalence chovat stejným způsobem

- Občas se v praxi můžete setkat s definicí: všechny hodnoty z konkrétní třídy ekvivalence mají stejnou šanci odhalit chybu

Tato definice není správně. Proč?

- Konkrétní hodnoty (data) pro test pak vybereme z vytvořených tříd ekvivalence
- Kolik jich vybrat? Záleží, jaké chceme mít **testovací pokrytí** (viz dále)

Třída ekvivalence

Jedná se zde o spojitý interval?
- Reprezentace čísla v počítači
- Definice maximální přesnosti hodnoty

Typy tříd ekvivalence – podle typu vstupu:

- **Interval**
  - Například pro: částka, věk, číslo účtu
  - Hranice intervalu = mezní podmínky (viz dále)
- **Diskrétní hodnoty**
  - Například pro: typ prohlížeče, metoda platby, položka v menu, výběr z položek v číselníku
  - Nejsou mezní podmínky

Typy tříd ekvivalence – podle validity dat:

- **Validní třídy ekvivalence** – nastavují situace, které systém zpracuje standardním způsobem. „Správná data“.
- **Nevalidní třídy ekvivalence** – nastavují situace, na které by měl systém zareagovat korektním ošetřením chyby. „Naschvál zadaná neplatná data“.

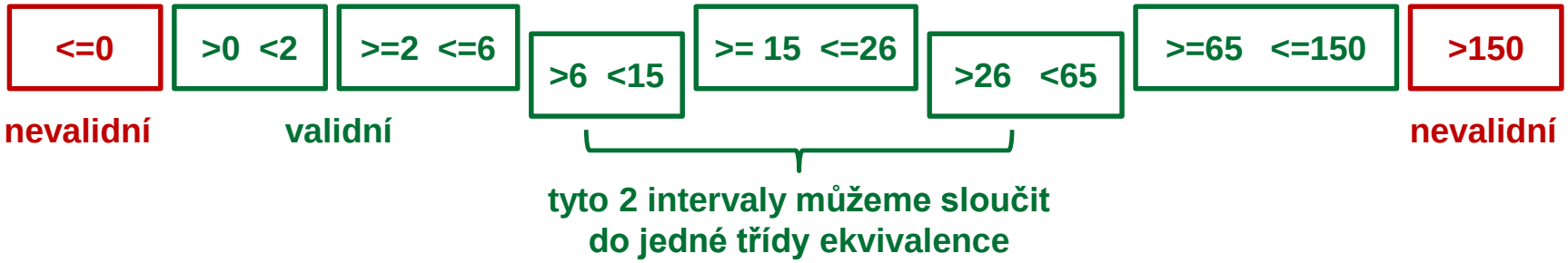
Mohou být nevalidní třídy ekvivalence i v typu „diskrétní hodnoty“?

Třídy ekvivalence - příklad

Interval

Systém pro rezervaci letenek – věk pasažéra:

Od 2 let musí mít pasažér vlastní sedačku. Do 6 let včetně může letět pouze s rodiči. V rozsahu od 15 do 26 let včetně dostává slevu v programu Euromládežník. Od 65 let včetně dostává slevu pro seniory. Maximální věk pro přepravu je 150 let.



Diskrétní hodnoty

Systém pro rezervaci letenek – typ platby:

Letenku je možné zaplatit hotově na letišti, převodem z účtu, kreditní kartou a pomocí PayPal.

cash, převod, kreditka, PayPal

Třída ekvivalence

Typy tříd ekvivalence – podle validity dat – jemnější dělení:

Typ třídy ekvivalence	Co znamená pro aplikaci	Příklad: políčko pro číslo účtu
Nevalidní třída ekvivalence z technického pohledu	Data, která neodpovídají datovému typu vstupu = neplatná data, která aplikace musí ošetřit, aby nezpůsobila pád	• Řetězec • Číslo, ale jako číslo účtu nevalidní formát
Nevalidní třída ekvivalence z business pohledu	Data, která sice odpovídají datovému typu vstupu, ale z pohledu specifikace business procesu jsou nevalidní	• Číslo účtu ve validním formátu, ale mimo nadefinovaný rozsah číselné řadu účtů • Číslo účtu ve validním formátu ve stejné bance, ale takový účet neexistuje
Validní třída ekvivalence	Platná data, která mají být zpracovávána podle business specifikace, vyvolávají korektní průběhy procesů v aplikaci	• Číslo účtu ve validním formátu v jiné bance, ale takový účet neexistuje • Existující číslo účtu ve stejné bance • Existující číslo účtu v jiné bance

Třída ekvivalence

Pravidla konzistence při určování tříd ekvivalence:

- Třída ekvivalence nesmí být prázdná (tj. určíme nějakou podmínku, která bude vymezovat data v ní, ale žádná data nebo případy jí nebudou odpovídat)
- Žádné dvě třídy ekvivalence nesmí mít neprázdný průnik
- Sjednocení všech tříd ekvivalence musí být původní
  - interval (pro typ interval)
  - množina (pro typ diskrétní hodnoty)

Alternativní použití:

- Použití tříd ekvivalence pro omezení počtu konfigurací

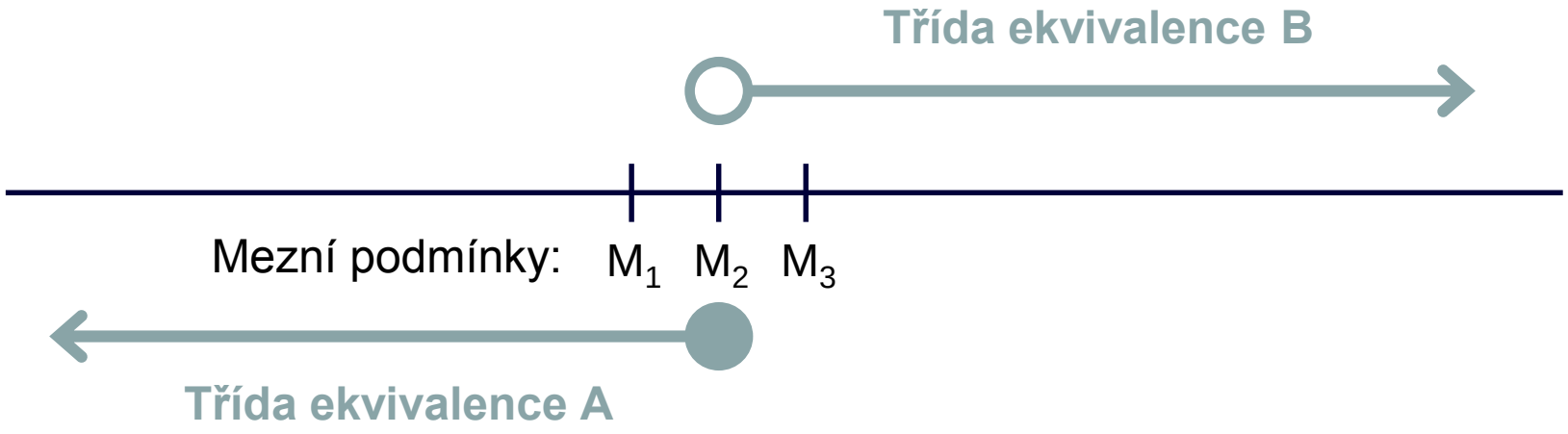
Jak určit třídy ekvivalence?

- U diskrétních hodnot: Analýza požadavků a specifikace
- U intervalu: Analýza požadavků a specifikace pomocí techniky **mezních podmínek**

Mezní podmínka

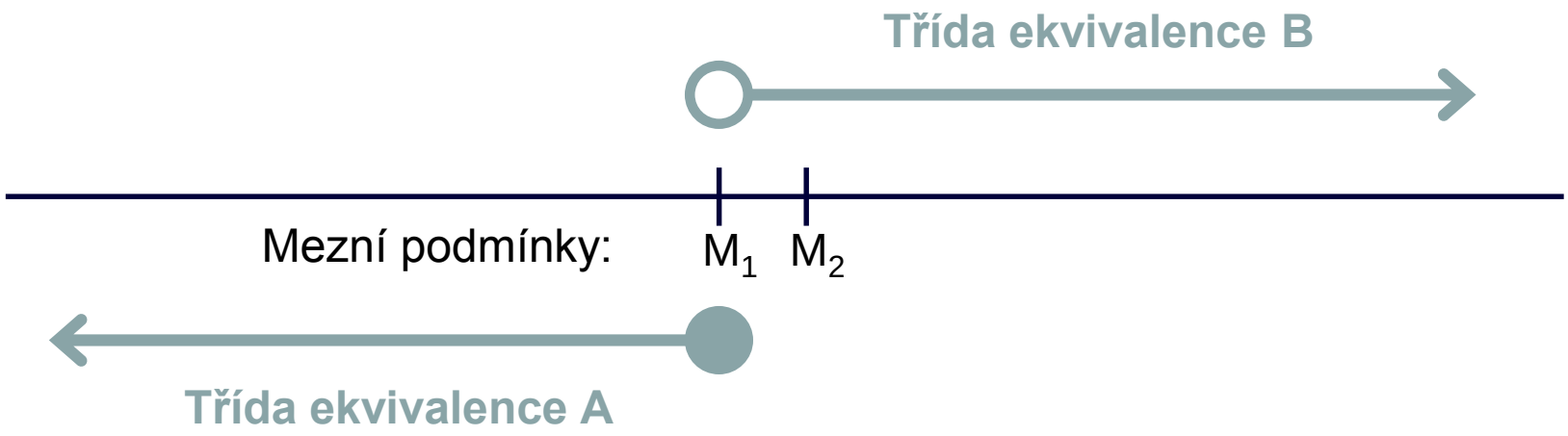
- ISTQB definice: Hodnota, která tvoří hranici mezi třídami ekvivalence, nebo je o malý inkrement posunutá na jednu ze stran této hranice

- Boundary value



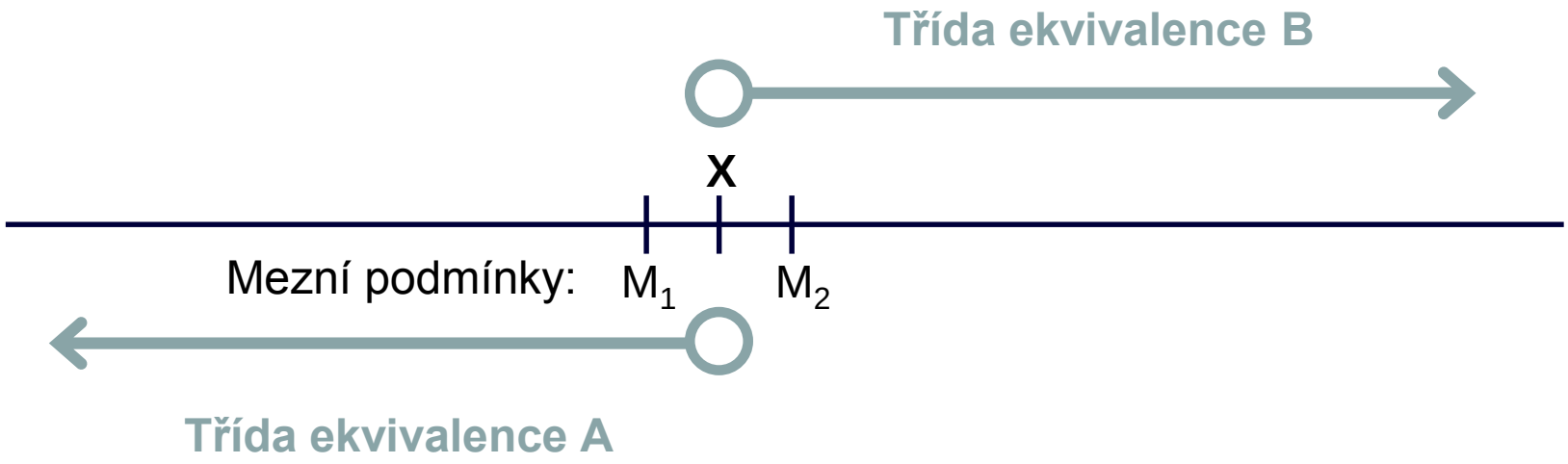
Mezní podmínka

- Alternativa: testujeme pouze 2 hodnoty kolem hranice třídy ekvivalence, ne 3
- Menší množství testovacích případů, ale potenciálně větší možnost úniku chyby



Mezní podmínka

- Jak se zachovat k hodnotě **X**, pokud třídy ekvivalence A a B vypadají podle specifikace takto?



Mezní podmínka

- Jak velký má být ten „malý inkrement“ z definice mezní podmínky?

- <u>Přesnost dané hodnoty</u>
  - Přesnost **daná datovým typem** *(Příklad: SQL typ DECIMAL(10,4) se 4 desetinnými místy)*
  - Přesnost **daná specifikací** *(Příklad: Částku transakce účtujeme se 2 desetinnými místy. Zaokrouhlovací rozdíly se řeší opravnými položkami.)*

- <u>Iterace kolem hranice třídy ekvivalence:</u>
  - M – mezní hodnota, tvořící hranici třídy ekvivalence
  - I – nenulová nejmenší hodnota, kterou jde uložit
  - Testujeme M, M + I, M - I

Mezní podmínky

Vazba mezních podmínek na typy tříd ekvivalence:

Typ třídy ekvivalence	**Kde vzít mezní podmínky**	**Jak je najít**	**Příklad: věk pasažéra**
Nevalidní třída ekvivalence z technického pohledu	Technické limity daného vstupu (někdy i specifikace)	Rozsah hodnot pro validní vstup Datový typ v programu Datový typ v databázi	• Políčko pro věk pasažéra má rozsah platné hodnoty 0-150
Nevalidní třída ekvivalence z business pohledu	Požadavky Specifikace	Informace ve specifikaci, že určité hodnoty (daný rozsah hodnot) nejsou povoleny nebo jsou ze zpracování vyloučeny	• Limit věku je 150 let
Validní třída ekvivalence	Požadavky Specifikace	Informace ve specifikaci, že od určité hodnoty (pro daný rozsah hodnot) dochází ke konkrétní variantě zpracování	• Pasažér od 2 let má vlastní sedadlo • Pasažér do 6 let musí letět s rodiči

Příklady na cvičeních

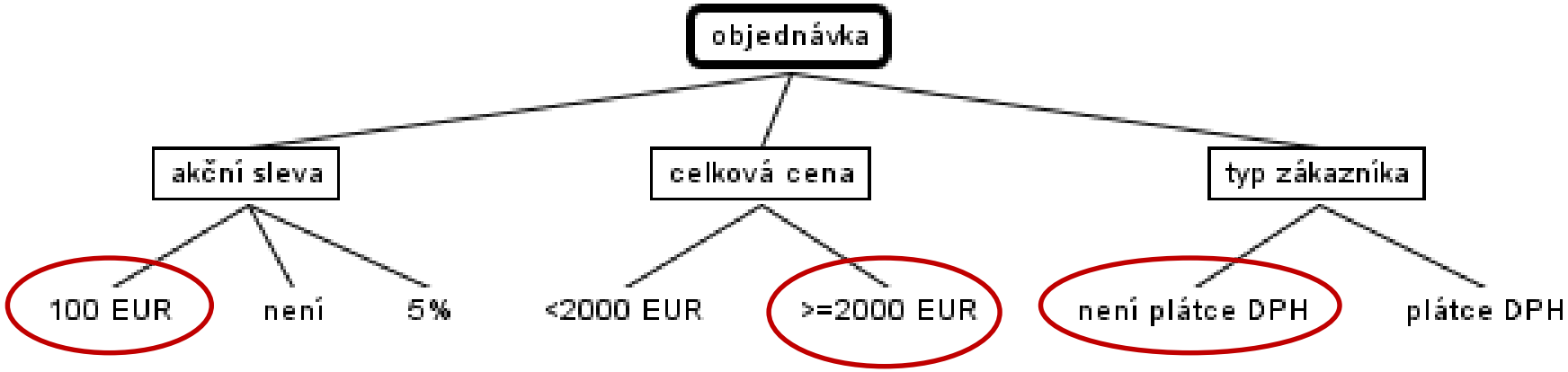
Mezní podmínka

Mezní podmínky pro technické limity vstupu:

- S vícevrstvou architekturou je analýza složitější:
  - **Klient**: Javascript nebo AJAX validace povolené hodnoty ve formuláři
  - **Webserver / aplikační server**: Data z HTTP requestu jsou v programu uložena v proměnné konkrétního typu
  - **Databázový server**: Data z programu uložena v SQL typu
  - **Integrační rozhraní**: Záleží na typu (DB, TXT, XML, RPC, webservice, messaging, …)
- Na každé vrstvě může být v principu jiný datový typ a rozsah hodnot, proto může dojít ke:
  - Špatné konverzi hodnoty *(např. přesnost u desetinných čísel, diakritika …)*
  - Přetečení rozsahu *(např. 300 uložených do 8 bitové proměnné šikovně přeteče na 44 …)*
  - Nežádoucímu chování, pokud se chybový výstup z jedné vrstvy zkonvertuje na platnou hodnotu na jiné vrstvě *(např. chyba se projeví jako null, null se zkonvertuje na 0 a ta se zpracovává jako platná hodnota …)*
  - …
- Konverze na integračních rozhraních je speciální kapitola

Kombinace vstupních hodnot

- Určili jsme třídy ekvivalence pro jednotlivé vstupy v rozhodovacím bodu v aplikaci.



Jak nyní určíme vhodné kombinace dat v jednotlivých vstupech rozhodovacího bodu, aby naše testy našly co nejvíce chyb (v limitech toho, jakou kapacitu na ty testy máme)**?**

Kombinace vstupních hodnot

Technika	Množství kombinací → testovací pokrytí	Co testujeme (jen pro orientaci - berte s rezervou!)	Pokrytí
MCC – Multiple Condition Coverage	Kompletní množina všech kombinací	Kritický SW – letadlo, lékařské přístroje, …	
MC/DC – Multiple Condition/Decision Coverage	Všechny kombinace, které mají vliv na výsledek rozhodovacího výrazu		
Pairwise testig (pokrytí je škálovatelné, pozice zde je pro kompletní kombinaci všech dvojic)	Výrazně menší počet kombinací, ale dle statistik stále odchytí většinu chyb	Důležité části nekritického SW: Jádro programu Výpočty, které musí být správně	
C/DC – Condition/Decision Coverage	Relativně malý počet kombinací pro testy střední / nízké intenzity	Nekritický SW – funkční testy	
CC – Condition Coverage			
DC – Decision Coverage			
Ruční výběr ještě menšího množství kombinací	Projití jen nejdůležitějších cest rozhodovacího bodu	Smoke test	

Některé techniky jsou vhodné pouze pro kombinace na základě logického výrazu (tj. v klasifikačním stromě jsou pro každý možný vstup právě dvě třídy ekvivalence – TRUE a FALSE). Které to jsou?

Kombinace vstupních hodnot

Logický výraz:

Pokud pilot **(** má nalétáno víc než 200 hodin **nebo** absolvoval víc než 10 linkových letů **) a zároveň** prošel výcvikem pro let nad mořem
může létat na zaoceánských letech

- Slovně napsané podmínky ve specifikaci (podmínky v pseudokódu, podmínky v UML diagramu atd.) jdou převést na logický výraz se spojkami AND, OR
- Ostatní logické spojky – konverze na výraz s AND, OR a negací
- Negace v důsledku jen přehodí podmínku

	podmínka A		**podmínka B**		**podmínka C**
IF (	letové hodiny > 200	**OR**	linkových letů > 10	**) AND**	prošel výcvikem

THEN může létat na zaoceánských letech

R = (A **OR** B) **AND** C

Anglická terminologie:
logický výraz = **decision**
podmínka = **condition**

DC, CC a C/DC

Test design technika	Princip	Minimální počet kombinací, pomocí kterých je možné otestovat (pro 2 třídy ekvivalence v podmínce – binární logika)
DC – Decision Coverage	Každý z výsledků celého logického výrazu je otestován alespoň jednou	2
CC – Condition Coverage	Každý z výsledků všech podmínek v logickém výrazu je otestován alespoň jednou	2
C/DC – Condition/Decision Coverage	Každý z výsledků všech podmínek v logickém výrazu a každý z výsledků celého logického výrazu je otestován alespoň jednou	2

Najdete protipříklad, kdy to pro nějaký výraz bude víc?

V čem je tedy rozdíl, když všechny produkují 2 kombinace?

MCC

Princip

- Všechny možné **kombinace výsledků podmínek** v logickém výrazu jsou testovány alespoň jednou

Počet kombinací

- pro logický výraz: $2^{(\text{počet podmínek})}$
- pro obecný klasifikační strom:

  (počet EC ve vstupu1) * (počet EC ve vstupu2) * (počet EC ve vstupu3) ...

  EC ... třída ekvivalence

- Nejvyšší úroveň pokrytí

MCC - příklad

IF **(** letové hodiny > 200 **OR** linkových letů > 10 **)** **AND** prošel výcvikem **THEN** může létat na zaoceánských letech

R = (A **OR** B) **AND** C

	Možné kombinace (test situations)							
	1	**2**	**3**	**4**	**5**	**6**	**7**	**8**
A	**1**	**1**	**1**	**1**	**0**	**0**	**0**	**0**
B	**1**	**1**	**0**	**0**	**0**	**0**	**1**	**1**
C	**1**	**0**	**0**	**1**	**1**	**0**	**0**	**1**
R	**1**	**0**	**0**	**1**	**0**	**0**	**0**	**1**

MCC - příklad

IF (letové hodiny > 200 **OR** linkových letů > 10) **AND** prošel výcvikem **THEN** může létat na zaoceánských letech

R = (A **OR** B) **AND** C

	Možné kombinace (test situations)							
	1	**2**	**3**	**4**	**5**	**6**	**7**	**8**
A	letové hodiny > 200	letové hodiny > 200	letové hodiny > 200	letové hodiny > 200	letové hodiny <= 200	letové hodiny <= 200	letové hodiny <= 200	letové hodiny <= 200
B	linkových letů > 10	linkových letů > 10	linkových letů <= 10	linkových letů <= 10	linkových letů <= 10	linkových letů <= 10	linkových letů > 10	linkových letů > 10
C	prošel výcvikem	neprošel výcvikem	neprošel výcvikem	prošel výcvikem	prošel výcvikem	neprošel výcvikem	neprošel výcvikem	prošel výcvikem
R	**může létat na z.l.**	**nemůže létat na z.l.**	**nemůže létat na z.l.**	**může létat na z.l.**	**nemůže létat na z.l.**	**nemůže létat na z.l.**	**nemůže létat na z.l.**	**může létat na z.l.**

MC/DC

Princip

- Každý z výsledků všech podmínek v logickém výrazu je otestován alespoň jednou a zároveň každá z podmínek nezávisle ovlivnila výsledek logického výrazu

  tím, že měníme pouze vstupní hodnoty pro tuto podmínku, zatímco vstupní hodnoty pro ostatní podmínky držíme zafixované

- Pracuje s termínem **neutrální hodnota**

Příklad použití

- DO-178B – Software Considerations in Airborne Systems and Equipment Certification
- Software, který má vliv na hladký průběh letu a přistání musí být otestován s pokrytím MC/DC

MC/DC – neutrální hodnota

Nejjednodušší situace

A	B	R = A AND B	Komentář
0	0	0	Pokud se v této situaci změní jakýkoliv ze vstupů A nebo B, výsledek R **se nezmění**
0	1	0	Pokud se v této situaci změní jeden ze vstupů A nebo B, změní se i výsledek R
1	0	0	
1	1	1	

Neutrální hodnota pro **AND** je **1**

A	B	R = A OR B	Komentář
0	0	0	Pokud se v této situaci změní jeden ze vstupů A nebo B, změní se i výsledek R
0	1	1	
1	0	1	
1	1	1	Pokud se v této situaci změní jakýkoliv ze vstupů A nebo B, výsledek R **se nezmění**

Neutrální hodnota pro **OR** je **0**

MC/DC

Způsob zápisu

A	B	R = A AND B
0	0	0
0	1	0
1	0	0
1	1	1

Použijeme tyto 3 kombinace

Výsledek výrazu

R = A AND B	1	0
A	1 1	0 1
B	1 1	1 0

Hodnota pro A

Hodnota pro B

Tato kombinace se zredukuje

MC/DC

IF **(** letové hodiny > 200 **OR** linkových letů > 10 **)** **AND** prošel výcvikem **THEN** může létat na zaoceánských letech

R = (A **OR** B) **AND** C

R = (A OR B) AND C	**1**	**0**
A	. . .	. . .
B	. . .	. . .
C	. . .	. . .

MC/DC

IF **(** letové hodiny > 200 **OR** linkových letů > 10 **)** **AND** prošel výcvikem **THEN** může létat na zaoceánských letech

R = (A **OR** B) **AND** C

R = (A OR B) AND C	**1**	**0**
A	1 . .	0 . .
B	. 1 .	. 0 .
C	. . 1	. . 0

MC/DC

IF **(** letové hodiny > 200 **OR** linkových letů > 10 **)** **AND** prošel výcvikem **THEN** může létat na zaoceánských letech

R = (A **OR** B) **AND** C

R = (A OR B) AND C	1	0
A	1 0 1	0 0 1
B	0 1 1	0 0 1
C	. . 1	. . 0

Doplňujte neutrální hodnoty, kde je možné jednoznačně určit výsledek výrazu

MC/DC

IF (letové hodiny > 200 **OR** linkových letů > 10) **AND** prošel výcvikem **THEN** může létat na zaoceánských letech

R = (A **OR** B) **AND** C

R = (A OR B) AND C	1	0
A	1 0 1	0 0 1
B	0 1 1	0 0 1
C	. . 1	. . 0

A OR B musí být 1

Možnosti:

1 1

0 1

1 0

A OR B může být 0 nebo 1

Možnosti:

0 0

0 1

1 0

1 1

MC/DC

IF (letové hodiny > 200 **OR** linkových letů > 10) **AND** prošel výcvikem **THEN** může létat na zaoceánských letech

R = (A **OR** B) **AND** C

R = (A OR B) AND C	1	0
A	1 0 1	0 0 1
B	0 1 1	0 0 1
C	. . 1	. . 0

tato kombinace se zredukuje

A OR B musí být 1

Možnosti:

1 1

0 1

1 0

Vybereme tak, aby se zredukovalo co nejvíc kombinací

A OR B může být 0 nebo 1

Možnosti:

0 0

0 1

1 0

1 1

Zde můžeme vybrat cokoliv **(pokud k tomu máme odpovídající informace, tak vybereme data, která mají větší důležitost pro test)**

MC/DC

IF (letové hodiny > 200 **OR** linkových letů > 10) **AND** prošel výcvikem **THEN** může létat na zaoceánských letech

R = (A **OR** B) **AND** C

R = (A OR B) AND C	1	0
A	1 0 1	0 0 1
B	0 1 1	0 0 1
C	0 1 1	0 0 0

tato kombinace se zredukuje

tato kombinace se zredukuje

MC/DC

IF **(** letové hodiny > 200 **OR** linkových letů > 10 **)** **AND** prošel výcvikem **THEN** může létat na zaoceánských letech

R = (A **OR** B) **AND** C

R = (A OR B) AND C	**1**	**0**
letové hodiny > 200	**1 0 1**	0 0 1
linkových letů > 10	**0 1 1**	**0 0 1**
prošel výcvikem	0 1 1	**0 0 0**

MC/DC

IF **(** letové hodiny > 200 **OR** linkových letů > 10 **)** **AND** prošel výcvikem **THEN** může létat na zaoceánských letech

R = (A **OR** B) **AND** C

Kombinace vstupů	**1 0 1**	**0 1 1**	**0 0 1**	**0 0 0**
letové hodiny	> 200	<= 200	<= 200	<= 200
linkových letů	<= 10	> 10	<= 10	<= 10
prošel výcvikem	ano	ano	ano	ne
výsledek	**může létat na z.l.**	**může létat na z.l.**	**nemůže létat na z.l.**	**nemůže létat na z.l.**

Pairwise testing

Potřebuji zredukovat počet kombinací vstupů pro test a pořád chci mít dobrou šanci odhalit chybu.

Princip

- Statistika: Pokud je někde v software defekt, aktivuje se nejčastěji buď
  - hodnotou jednoho konkrétního vstupu
  - kombinací hodnot ve dvou vstupech
- Situace, kdy se defekt aktivuje kombinací hodnot tří a více vstupů jsou méně časté
- Vyberu konkrétní dvojice (**pairwise testing**) nebo trojice (**thereewise testing**) vstupů a v nich pokryji všechny kombinace tříd ekvivalence, které do nich budu zadávat

Počet kombinací

- Podle konfigurace párů nebo trojic a počtu tříd ekvivalence zadávaných do jednotlivých vstupů

Pairwise testing

Funguje to?

- **Statistické pozorování účinnosti párového testování:**

- D. R. Wallace and D. R. Kuhn, 2001:

  Evaluating FDA recall class failures in medical devices we established that] [...] out of the 109 reports that [were] detailed [enough], 98% showed that the problem could have been detected by testing the device with all pairs of parameter settings.

- K. Burr and W. Young, 1998:

  Our initial trial of this was on a subset Nortel's internal e-mail system where we able cover 97% of branches with less than 100 valid and invalid testcases, as opposed to 27 trillion exhaustive testcases.

(převzato z pairwise.org)

Pairwise testing

Škálovatelnost

Jak se změní počet kombinací a pokrytí?

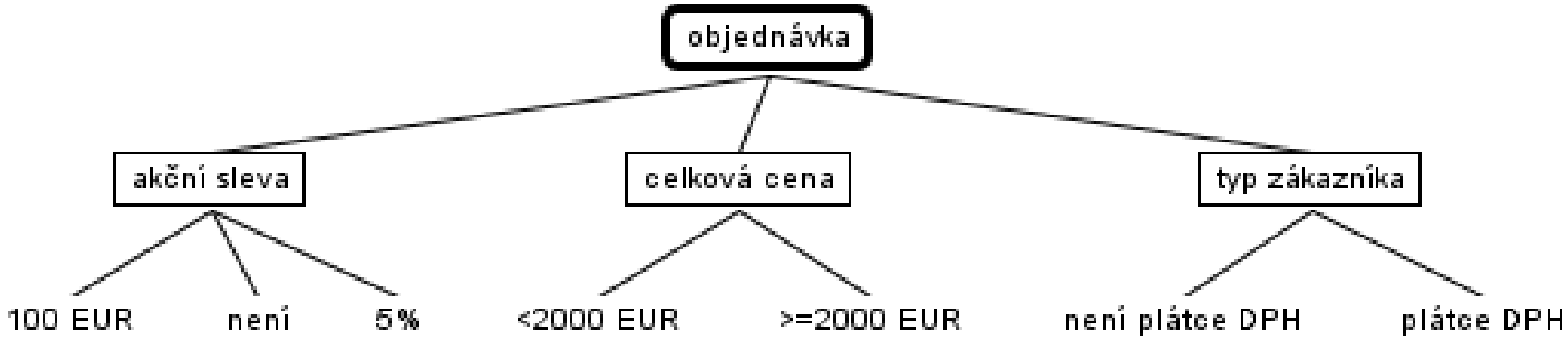
- Dvojice vytvořené pro všechny možné kombinace vstupů
- Dvojice vytvořené jen pro nejdůležitější vstupy - jak je vybrat?
  - Technická priorita nebo priorita z pohledu business procesu
  - Předchozí chybovost
  - …

Podle čeho dalšího?

- Použití trojic místo dvojic
- Trojice a dvojice jde kombinovat zároveň, podle důležitosti vstupů

Pairwise testing

Příklad: klasifikační strom pro vstupy:

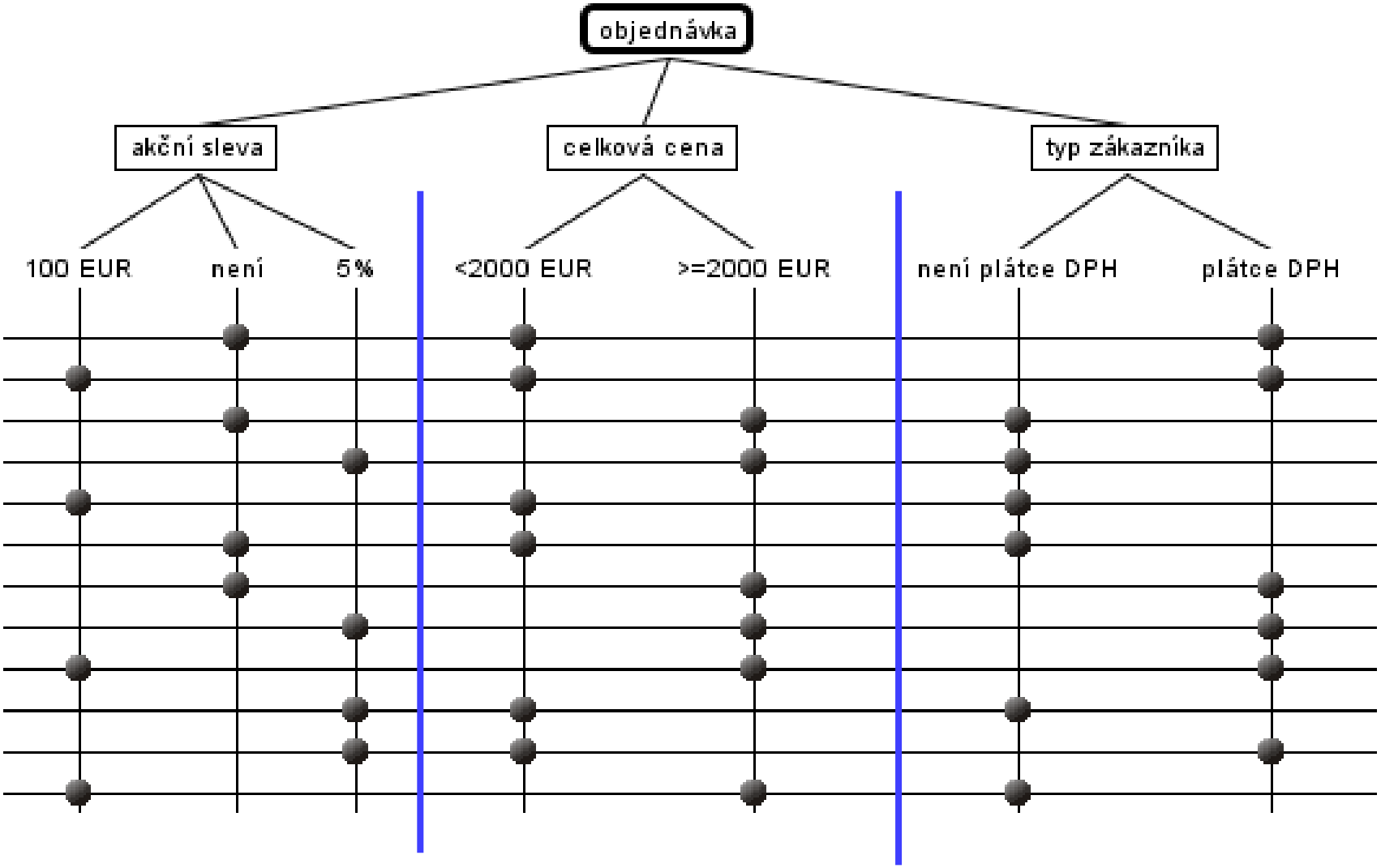


Pokud bychom testovali všechny kombinace:

Počet kombinací =
(počet EC v prvním vstupu) * (počet EC v druhém vstupu) * (počet EC v třetím vstupu)
= 3 * 2 * 2 = 12

Pairwise testing

Kompletní pokrytí všemi možnými kombinacemi – 12 variant k otestování:



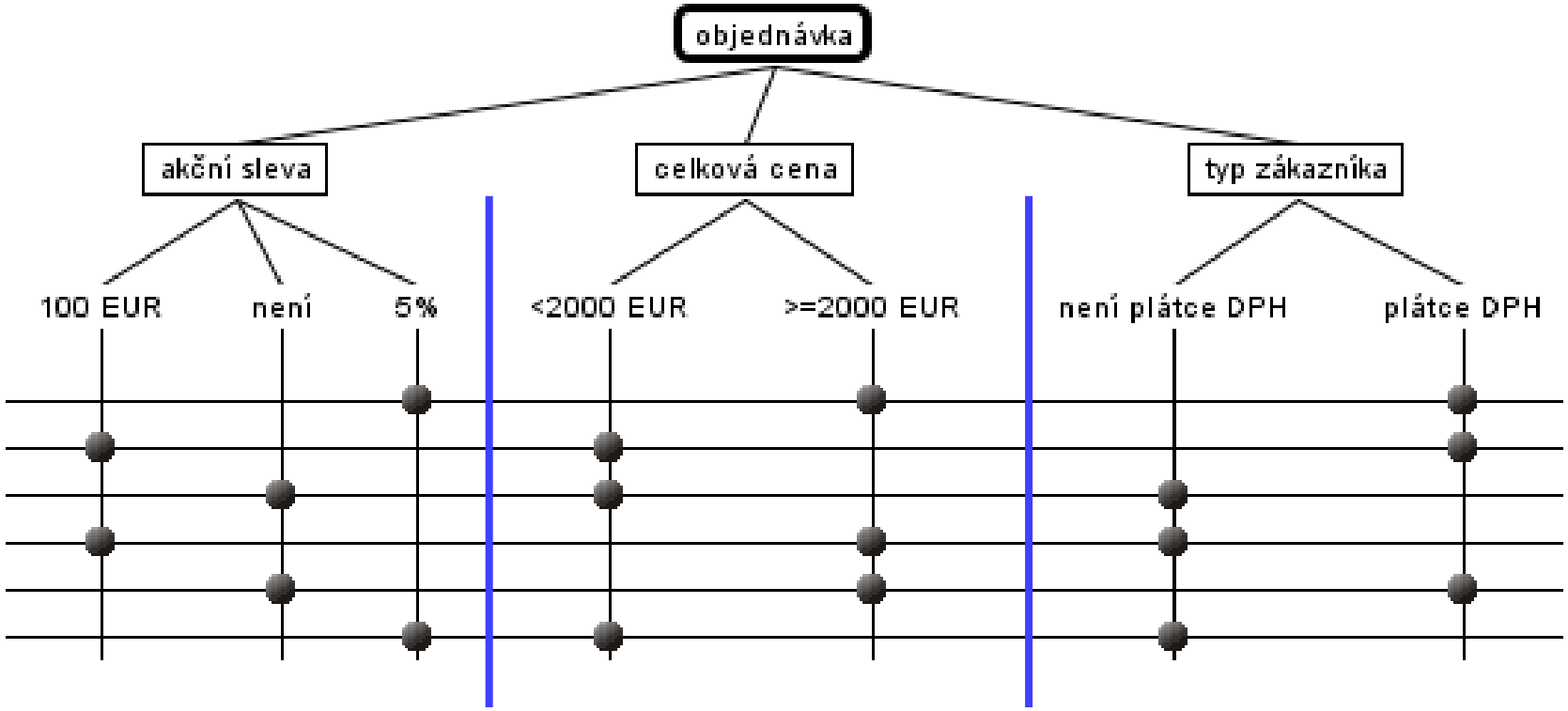
Pairwise testing

Kompletní pokrytí všemi možnými kombinacemi – 12 variant k otestování:

akční sleva	celková cena	typ zákazníka
není	<2000 EUR	plátce DPH
100 EUR	<2000 EUR	plátce DPH
není	>=2000 EUR	není plátce DPH
5%	>=2000 EUR	není plátce DPH
100 EUR	<2000 EUR	není plátce DPH
není	<2000 EUR	není plátce DPH
není	>=2000 EUR	plátce DPH
5%	>=2000 EUR	plátce DPH
100 EUR	>=2000 EUR	plátce DPH
5%	<2000 EUR	není plátce DPH
5%	<2000 EUR	plátce DPH
100 EUR	>=2000 EUR	není plátce DPH

Pairwise testing

V každém páru vstupů jsou pokryty všechny kombinace – 6 variant k otestování:



Pairwise testing

V každém páru vstupů jsou pokryty všechny kombinace – 6 variant k otestování:

akční sleva	celková cena	typ zákazníka
5%	>=2000 EUR	plátce DPH
100 EUR	<2000 EUR	plátce DPH
není	<2000 EUR	není plátce DPH
100 EUR	>=2000 EUR	není plátce DPH
není	>=2000 EUR	plátce DPH
5%	<2000 EUR	není plátce DPH

Pairwise testing

Nástroje pro vytváření kombinací:

- Allpairs (James Bach)
- CTE XL (Berner & Mattner Systemtechnik)

- Dle pairwise.org je k dispozici 37 nástrojů :)
  http://www.pairwise.org/tools.asp

Výběr jen několika kombinací

Jak dosáhnout účinných testů, ikdyž budeme testovat jen několik kombinací (tj. testovací pokrytí bude malé)?

Zaměření na reálná rizika:

- Hlubší analýza mezních podmínek
- Přidání důležitých průchodů z business analýzy do připravených testovacích scénářů / dat
  - Hlavní pozitivní průchod
  - Nejčastější průchod
  - Průchody na které se váže největší objem obchodní aktivity generované systémem (nemusí být nejčastější ani nejočekávanější průchod !)

- **Toto platí obecně.** Při použití konkrétní testdesign techniky je vhodné zrevidovat, že obsahuje hlavní pozitivní průchod celým workflow a statisticky nejčastěji očekávané kombinace vstupů (aby shodou okolností nebyly mimo pokrytí)

Jak to udělat?

I SPENT THE WEEK WRITING A TEST SCRIPT FOR OUR PRODUCT.
Dilbert.com DilbertCartoonist@gmail.com
AND I WROTE A TEST SCRIPT TO TEST DILBERT'S TEST SCRIPT.
3-24-11 ©2011 Scott Adams, Inc./Dist. by UFS, Inc.
YOUR SCRIPT WAS ALMOST PERFECT. KEEP UP THE GOOD WORK, BUDDY.

Děkuji za pozornost

buresm3@fel.cvut.cz

Zdroje

Pairwise testing:
http://www.pairwise.org/
K. Burr and W. Young: Combinatorial Test Techniques: Table-based Automation, Test Generation and Code Coverage, 1998

TMAP Next:
http://www.tmap.net/en/tmap-next

Allpairs, James Bach:
http://www.satisfice.com/tools.shtml

CTE XL:
http://www.berner-mattner.com/en/berner-mattner-home/products/cte

ISTQB Certified tester Foundation level syllabus, release 2011
http://www.istqb.org/downloads/viewcategory/16.html

ISTQB Glossary of testing terms, version 2.2
http://www.istqb.org/downloads/viewcategory/20.html